DarnerView: Journal of Artificial Intelligence

Year 2026 • Manuscript submitted for review
e-ISSN: assigned by the journal
https://www.darnerview.com

DOI: to be assigned by the journal

RESEARCH ARTICLE

Dependency Change Signals and Second Agent Auditing in a Controlled Scheduling Simulation

Sepide Ebrahimian Maqsoud[1], **Moslem Mohseni Khah**[2*], **Navid Sadeghi**[3]

[1] Department of Electrical Engineering, Hamedan University of Technology, Hamedan, Iran
[2] Department of Computer Engineering, Islamic Azad University, Hamedan Branch, Hamedan, Iran
[3] Department of Electrical Engineering – Control, Shahid Beheshti University, Tehran, Iran

**Corresponding author: Moslem Mohseni Khah • Email: moslem.mohsenikhah@iau.ir*

Received: 26.10.2026 • Accepted: — • Published: —

ABSTRACT

Calling a second agent after a planning disruption can improve repair reliability, but its cost may exceed its value. This simulation study tests whether structural changes identify repairs worth auditing. A locally generated scheduling environment uses twelve-task instances, an exact feasibility validator and a bounded deterministic constraint-programming reference. Algorithmic repair and audit agents overlook constraint checks under explicit parametric error models; no language model supplies the reported outcomes. At the default slip rate, single-agent feasibility was 0.832 and audited feasibility was 0.990, at 1.78 times the assumed token cost. The dependency-load feature had area under the receiver operating characteristic curve 0.572. Thresholds are tuned on disjoint development instances, with cluster-level uncertainty and multiplicity correction for a fixed utility-comparison family. Added critical-path and resource-pressure proxies test specific structural alternatives. The largest selected gain across the exploratory price grid and error models was 0.0004 utility units. The evidence does not support a useful dependency-change trigger under these assumptions. A validator-triggered audit can be preferable when validation is inexpensive. This negative result concerns the testbed and error models; it does not establish that structural routing fails for real language-model agents.

Keywords: multi-agent systems; LLM agents; collaboration trigger; dynamic scheduling; negative result; simulation study

© 2026 The Author(s). This article is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0): https://creativecommons.org/licenses/by-nc/4.0/

1. Introduction

A second agent is the most common remedy when one language-model agent is not reliable enough. Reviewers, verifiers and debaters improve accuracy, but they also multiply tokens and latency. Failure analyses of multi-agent systems show that more agents do not remove failures and can add coordination errors of their own (Cemri et al., 2025), and a large controlled study found that the best architecture depends on how well coordination matches the structure of the task (Kim et al., 2026). The practical question is therefore not whether a second agent helps, but when it is worth its cost.

Recent routing studies select collaboration topology before a task (Hong, 2026) or delegation format before a handoff (Banerjee & Chadha, 2026). Neither tests the post-disruption second-agent trigger studied here. Dependency changes may propagate repair errors, suggesting that their extent could help allocate audits. We test the hypothesis that *the amount of dependency change predicts erroneous single-agent repairs, and threshold-based auditing is more efficient than always auditing, never auditing, or random auditing at the same calling rate.*

The intuition is plausible and, as we show, only weakly supported by data from a controlled testbed. We report the study as a measurement of a decision signal, not as a method paper, and structure it around three questions:

1) RQ1: How well do features of the constraint difference predict that a single agent's repair will be infeasible?
2) RQ2: Does a trigger on those features beat always-single, always-team and a randomly mixed policy with the same invocation rate, under an explicit token-price utility?
3) RQ3: How does a failure-triggered rule that relies on an external validator compare, and at what validator cost does the comparison reverse?

2. Literature Review

2.1. Adaptive collaboration in multi-agent systems

The difficulty-aware topology selector of Hong (2026) predicts the success probability of each topology for a code problem and picks the best predicted success minus a tunable cost penalty. At 40% of the cost of always using a hierarchical team it reports higher pass@1. Routed Graph Handoff selects, before each delegation, whether to send a typed dependency graph or free text, reporting large savings in tokens (Banerjee & Chadha, 2026). Both adapt collaboration to task content, but their decisions occur at different boundaries. Routed Graph Handoff uses a single prompt-based classifier rather than a learned routing model, and its limitations acknowledge coarse task-type routing and unresolved execution-time headroom. Our setting differs in two ways: the decision is taken after a disruption, and the available signal is the difference between the dependency graphs before and after it.

2.2. Dynamic scheduling and rescheduling

Rescheduling after disruptions is a mature topic in operations research (Vieira et al., 2003), and resource-constrained project scheduling is NP-hard in general (Blazewicz et al., 1983). REALM-Bench proposes fourteen adjustable scheduling challenges for evaluating language-model agents and teams under disruptions (Geng & Chang, 2025). We use instances that follow the same ideas, precedence constraints, shared resources and disruptions, but generate them ourselves so that a bounded reference solution can be computed with a constraint-programming solver (Perron & Furnon, 2025). The study is not run on REALM-Bench itself.

2.3. Verification after failure

Wrapping tool calls with postcondition checks reduces unnecessary executions while keeping task success (Mansoor et al., 2026). The same idea underlies a failure-triggered collaboration rule: act, validate, and call help only when validation fails. Such a rule needs a validator, which is not always available for open-ended work. Part of our contribution is to quantify the validator cost at which the rule stops being preferable to a rule based on the structure of the change.

3. Materials and Methods

3.1. Research design

The study is a computational experiment with a development and a test split. Instances are generated from seeds; the development split (100 base instances per family) is used only to tune the trigger threshold, and the test split (200 base instances per family, disjoint seeds) is used for every reported number. Each base instance receives three disruptions, each repair is repeated 5 times with independent agent noise, and all policies are evaluated on the same episodes. Development base seeds start at 0, test seeds at 1000; the wide-family offset is 100000. No human participants or language-model calls are involved.

3.2. Scheduling instances and disruptions

Each instance has 12 tasks with integer durations of 1 to 5, a precedence graph, and two renewable resources with task demands. The *pipeline* family is a four-layer graph with light resource use; the *wide* family has a shallow graph and tight resource contention. The baseline schedule is a feasible incumbent of a CP-SAT model (Perron & Furnon, 2025). Three disruptions are applied to the instance:

- **Local:** the duration of a task with at most two transitive successors grows by one or two units.
- **Dependency:** two to four new precedence edges or mutual-exclusion pairs are added.
- **Cascade:** five to seven such constraints are added. This type, and the load-dependent error condition below, were added after a pilot showed small effects with the first two types, so they are exploratory extensions.

The *affected region* is the set of tasks touched by the disruption plus all their descendants. A feasible repair is a schedule that satisfies every constraint of the disrupted instance, checked by an exact validator; the reference makespan is the CP-SAT incumbent for the disrupted instance.

The reference solver uses one worker, a fixed random seed and a deterministic-time budget of 0.1 units per solve, with a serial feasible hint and resource/path lower bounds. Every result records solver status and lower bound; uncertified incumbents are never described as optima. A serial fallback retains hard seeds if no solver incumbent is available. Makespan deviations are measured relative to the incumbent and can be negative; they are not certified optimality gaps.

3.3. Repair agent and audit agent

The first agent re-places the tasks of the affected region in topological order. For each task it performs explicit constraint checks: one per predecessor, one per resource demanded for each candidate start time, and one per exclusion partner. Each check is *overlooked* with probability ε, and an overlooked check means the constraint is ignored, which can leave the schedule infeasible. The number of checks grows with the size of the affected region, so the probability of at least one harmful slip grows with it. Three error-model families are used. The default is a per-check slip probability $\varepsilon = 0.02$, with $\varepsilon = 0.005$, 0.01 and 0.04 as variations. A *load-dependent* model multiplies ε (0.005) by $1 + 0.5 \times$ region size, representing an agent whose reliability falls as more tasks must be handled at once. A *size-independent* model gives a fixed probability 0.30 of exactly one overlooked check per repair, as a control fixing slip occurrence rather than the probability that a slip causes infeasibility.

The second agent audits the first agent's schedule by re-checking every constraint of every region task. Each audit check is itself overlooked with probability ε. Flagged tasks and their descendants in the region are re-placed by the first agent, and the audit is repeated once. Tokens are counted from the checks performed: 300 per repair run, 200 per audit round, 25 per check, 100 per fix round and 150 for an external validator call.

3.4. Policies and trigger

- **Always single:** the first agent only.
- **Always team:** the first agent followed by the audit.
- **Dependency trigger:** call the audit if a feature of the constraint difference reaches a threshold θ tuned on the development split. Features are *load* (the number of constraints that must be re-checked in the affected region), *region size* and *number of changed constraints*; all are computed from the constraint difference alone.
- **Random matched:** call the audit with the same overall probability as the trigger, evaluated as an exact expectation.
- **Failure-triggered:** validate the first agent's schedule with an external validator and call the audit only if it fails.
- **Oracle:** call the audit only when the first repair is infeasible, at no validator cost (an unattainable bound).

Policies are compared on a utility that prices tokens against feasibility:

$$U = P(feasible) - \lambda \cdot tokens \quad (1)$$

where λ is the feasibility value of one token. The reported λ values are 1/20,000, 1/10,000, 1/5,000 and 1/2,500 per token; the headline value 1/5,000 (1,000 tokens equal 0.2 feasibility) was fixed before looking at test results. A second agent is worth calling on an episode only if the failure probability it removes exceeds its cost:

$$P(F \mid x) \cdot \Delta_feas > \lambda \cdot E[tokens_audit \mid x] \quad (2)$$

3.5. Statistical analysis

Predictive value is measured by the area under the ROC curve (AUC), computed with the Mann-Whitney statistic. Utilities are reported with 95% cluster-bootstrap intervals that resample base instances (1,000 resamples), because the repetitions and disruptions of one base instance are dependent (Efron, 1979; Field & Welsh, 2007). Differences between the trigger and the random-matched policy are paired by episode. The simulation provides no population, so intervals describe sampling variability across generated instances.

3.6. Ethical approval

The study uses simulated data only and involves no human participants or personal data; ethics approval is not required.

3.7. Primary family and additional structural features

The primary outcome is utility at the inherited headline token price and default per-check slip rate. The revised primary family contains seven paired cluster-level contrasts: each of five feature triggers against rate-matched random invocation, and the load trigger against always-single and

always-team. Cluster-level sign-flip tests use 5000 permutations; Holm correction covers all seven contrasts (Holm, 1979). Mean-difference intervals use 2000 cluster resamples and are nominal, not simultaneous. The family is specified for this revision after prior exploratory work and is not retrospectively preregistered. Feasibility, costs, AUC and price-grid maxima are secondary descriptive or exploratory summaries.

Two reviewer-motivated features were fixed before the new run: the fraction of precedence-critical tasks inside the affected region, and duration-weighted resource pressure summed over that region. Criticality uses precedence-only earliest and latest starts and ignores resource/exclusion constraints; it is a proxy, not an exact resource-constrained critical path. AUC intervals resample complete base instances with 500 replicates. Thresholds are selected on development instances and frozen on test instances. A constant-probability slip control fixes the chance of omitting a check, not the chance that an omission causes infeasibility.

3.8. Separate model-driven validation protocol

A separate held-out harness requests scheduling repairs and independent audits from a language model, using public twelve-task constraints, exact domain/feasibility validation and a bounded CP-SAT reference. Development and test streams are disjoint. Cached raw completions, prompt hashes and seed templates are retained. The client exposes a provider-independent interface with a hard per-paper budget cap. Provider discovery found no credentials or ready local model, and the official local-runtime download attempt did not complete. A labelled mock run tests the harness only; no real-model result is inferred.

4. Results

4.1. What the second agent buys

Table 1. *Single-agent and audited repair by disruption type (ε = 0.02, test split)*

Disruption	Episodes	Load	Region size	Feasible, single	Feasible, team	Tokens, single	Tokens, team
Local	2,000	3.5	1.4	0.888	0.992	654	1064
Dependency	2,000	14.4	4.5	0.827	0.990	1007	1795
Cascade	2,000	23.0	6.6	0.780	0.988	1326	2469

Note. Load is the mean number of constraints to re-check in the affected region. Feasible = share of repairs satisfying all constraints.

At ε = 0.02, the single agent repaired 83% of the 6,000 test repair episodes feasibly, and the audited pair 99% (Table 1). The audit raised tokens from 996 to 1776 per disruption. Among feasible repairs, mean makespan was 9.4% above the CP-SAT incumbent for the single agent and 10.3% for the team, so the evidence supports feasibility improvement without a demonstrated makespan advantage. Feasibility fell with the size of the disruption: 89% (local), 83% (dependency) and 78% (cascade) for the single agent.

4.2. Can the constraint difference predict failure?

Table 2. *Predictive value (AUC) of constraint-difference features for single-agent failure*

Error model	Failure rate	AUC, load	AUC, region size	AUC, changed constraints
Per-check, ε = 0.005	0.049	0.580	0.596	0.605
Per-check, ε = 0.01	0.093	0.575	0.589	0.601

Error model	Failure rate	AUC, load	AUC, region size	AUC, changed constraints
Per-check, ε = 0.02	0.168	0.572	0.587	0.594
Per-check, ε = 0.04	0.300	0.587	0.604	0.604
Load-dependent (ε_0 = 0.005)	0.151	0.671	0.686	0.665
Size-independent (p = 0.30)	0.102	0.442	0.459	0.460

Note. AUC = 0.5 means no predictive value. Computed on the test split (6,000 episodes per error model).

Features of the constraint difference carried a modest but consistent signal under per-check slips, with AUC between 0.57 and 0.60 (Table 2). When error rates themselves depended on region size, the AUC rose to 0.67–0.69. In the size-independent control it was 0.44–0.46, below 0.5: a single overlooked check is more likely to be decisive when the region is small and every check matters, so larger regions are slightly safer in that model. Predictive value is conditional on how check opportunities and their consequences depend on structure; this simulator does not establish a universal weakness of structural signals.

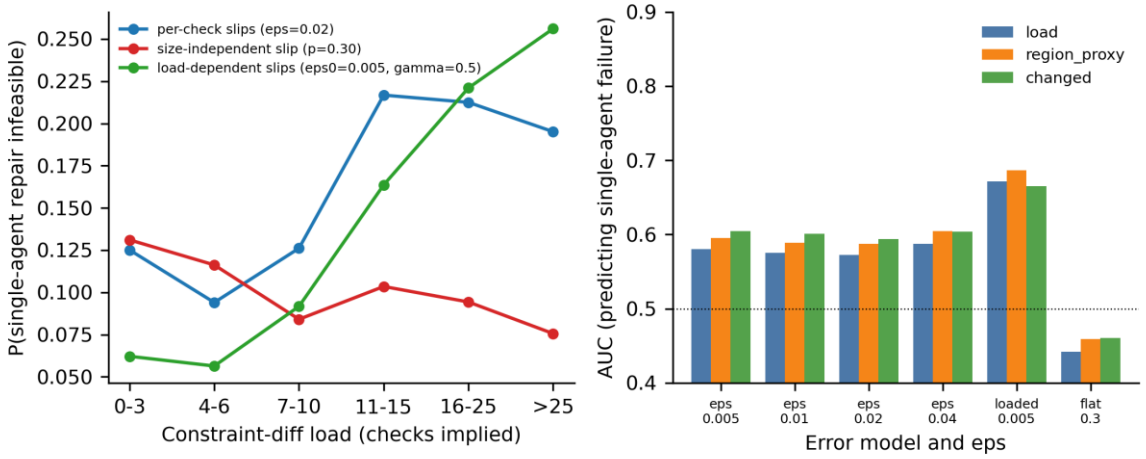


Figure 1. *Single-agent failure against the constraint-difference load (left) and AUC of the three features under each error model (right).*

4.3. Does a trigger beat the corner policies?

Table 3. *Policies at the headline token price (λ = 1/5,000, ε = 0.02, test split)*

Policy	Feasible	Tokens	Audit rate	Utility [95% CI]
Always single	0.832	996	0.000	0.633 [0.615, 0.649]
Always team	0.990	1776	1.000	0.634 [0.624, 0.645]
Dependency trigger (load)	0.990	1774	0.993	0.635 [0.624, 0.645]
Random, same rate	0.988	1770	0.992	0.634 [0.623, 0.645]
Failure-triggered (150-token validator)	0.990	1484	0.168	0.693 [0.680, 0.705]
Oracle failure signal	0.990	1334	0.168	0.723 [0.709, 0.736]

Note. The trigger threshold was tuned on the development split. Audit rate is the share of disruptions for which the second agent was called.

The tuned trigger on the load feature called the audit for 99% of disruptions, so it was practically equivalent to always-team (Table 3). At the three other token prices the tuned threshold likewise selected an extreme (never or almost always). The utility difference to a random policy with the same calling rate was 0.0003 [0.0001, 0.0005], which is small; multiplicity-adjusted inference for the fixed primary family is reported separately below. Across a fine grid of 40 token prices, the

best gain of the trigger over the better of always-single and always-team was 0.0003 [0.0001, 0.0005] at ε = 0.02, 0.0004 at ε = 0.04, 0.0003 in the load-dependent condition and 0.0002 in the size-independent control (Figure 2). These gains come from skipping the audit for the small share of disruptions with almost no constraints to re-check.

The reason is visible in the second equation. The audit costs 780 tokens and removes about 16 percentage points of failure on average. To justify the call for a subgroup, that subgroup's failure probability must exceed the break-even level for the token price. For most prices, either every subgroup is above that level (call always) or every subgroup is below it (never call); the features do not separate subgroups widely enough to produce a threshold between the two.

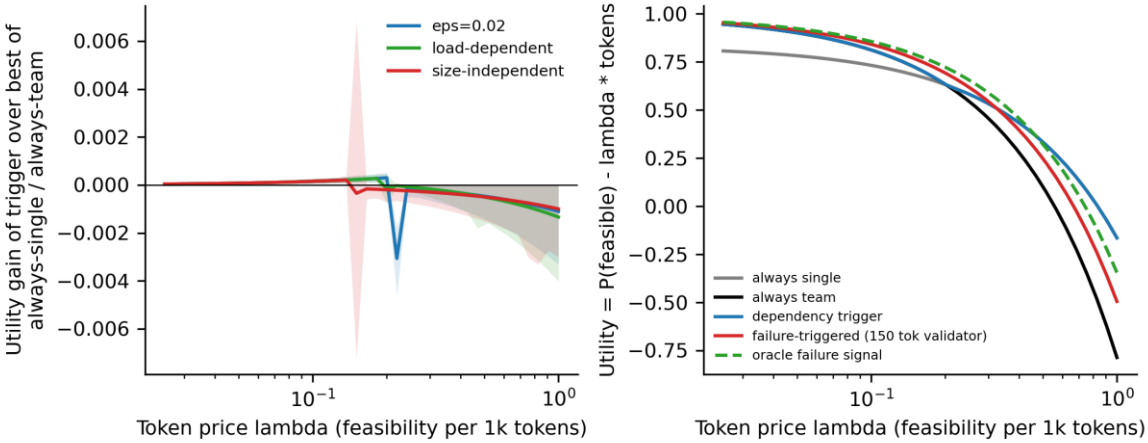


Figure 2. *Left: utility gain of the tuned trigger over the better corner policy across token prices. Right: utility of all policies at ε = 0.02.*

4.4. Failure-triggered rule and validator cost

With a validator costing 150 tokens, the failure-triggered rule reached a utility of 0.693 [0.680, 0.705], against 0.635 [0.624, 0.645] for the dependency trigger and 0.723 [0.709, 0.736] for the unattainable oracle. The advantage of the failure-triggered rule disappeared at about 440 tokens per validation (supplementary validator-cost analysis), beyond which a validator costs more than it saves by sparing audits. A dependency-based trigger would be relevant only in the narrow region of no available validator and strongly load-dependent errors; in this study it was not better there either.

The solver certified optimality for 98.2% of default-setting test disruptions. The mean incumbent-to-lower-bound relative gap was 0.002. Feasibility is judged exactly, but makespan comparisons must be interpreted relative to these bounded incumbents rather than universal optimum certificates.

4.5. Additional features and adjusted primary comparisons

Table 4. *Exploratory structural features and the fixed primary utility comparisons*

Feature	Failure AUC and interval	Utility difference and interval	Holm p
load	0.572 [0.551, 0.593]	0.0003 [0.0001, 0.0005]	0.0154
region proxy	0.587 [0.566, 0.606]	0.0000 [0.0000, 0.0000]	1.0000
changed	0.594 [0.571, 0.615]	0.0000 [0.0000, 0.0000]	1.0000
critical overlap	0.571 [0.546, 0.597]	0.0000 [0.0000, 0.0000]	1.0000

Feature	Failure AUC and interval	Utility difference and interval	Holm p
resource pressure	0.640 [0.618, 0.663]	0.0016 [−0.0004, 0.0036]	0.5169

Note. Primary contrasts compare each trigger with exact random invocation at its test calling rate. Holm correction includes both additional fixed-corner comparisons in the supplementary primary family. AUC intervals are exploratory cluster-bootstrap intervals.

The added features are reported in Table 4 without selecting the strongest test feature for a revised headline. This addresses RQ1 and RQ2 beyond the original coarse features. Price-grid maxima remain selected exploratory summaries and their pointwise intervals do not account for maximisation across prices. Neither a non-significant result nor a small observed effect proves equivalence.

The load-trigger gain over matched random was statistically detectable after correction (Holm p = 0.0154), but its magnitude was only 0.0003 utility units. Against always-team, the gain was 0.0003 [0.0001, 0.0005]. This is a small-effect result, not a claim that every contrast is null.

4.6. Study 2 execution status

No real language-model repair or audit was run. The separate mock-only stream checks public task construction, parsing and exact scoring. Its success/failure labels are not scientific results and no model confidence interval is reported. The negative result therefore concerns the assumed repair and audit mechanisms, conditional on this testbed.

5. Discussion

The hypothesis that the amount of dependency change after a disruption identifies the repairs that need a second agent is not supported in this testbed. The signal is present, in line with the intuition that larger changes require more checks, but it is weak, and its strength is determined by a property of the agent that the constraint difference cannot observe: whether the agent's reliability declines with load. Where we imposed such a dependence, AUC rose to about 0.69, which is useful for ranking but still insufficient to move the optimal policy away from the corners.

The negative result is informative for the design of collaboration policies. First, it suggests that the economic decision is dominated by the second agent's average value and cost, and that routing on coarse structural features adds little unless the base error rate is strongly heterogeneous across tasks. This is consistent with reports that coordination benefits depend on task structure and on the capability of the individual agent (Kim et al., 2026), and it complements the selection of topology at the task or delegation boundary (Hong, 2026; Banerjee & Chadha, 2026) by measuring a different signal in a controlled post-disruption testbed; predictive strengths across these tasks are not directly comparable. Second, whenever a cheap validator exists, the rule that calls for help only after a failed validation was better than any structural rule we tried, and the free-signal oracle is an optimistic reference for attainable utility. Third, our results show a practical way to decide: measure the audit's marginal failure removal, compute the break-even failure probability for the token price, and check whether any observable feature separates episodes above and below it.

The study design contains the usual risks of simulation work. The error model is mechanistic but assumed: independent overlooked checks. Real agents fail through misreading, hallucinated constraints and loss of context, which may correlate with structure in ways the model does not represent. The load-dependent condition and the cascade disruption were added after the pilot

because the first results were small, so they should be treated as exploratory, and the confirmatory claim is limited to the pre-specified conditions.

The absence of real-model validation limits transfer: actual scheduling errors can correlate with semantic ambiguity, context length or shared misconceptions rather than independent skipped checks. The additional structural features test specific alternatives, not the space of all useful routing signals. A real-model result could strengthen the signal and would require changing the conclusion.

6. Conclusion

In a controlled scheduling testbed, a single agent's repair was infeasible in 17% of disruptions, a second agent's audit removed most of those failures at 1.78 times the tokens, and features of the dependency change predicted the failures only weakly. A threshold trigger on those features offered a negligible gain over the better of always-single and always-team, and a failure-triggered rule with a validator was preferable below about 440 tokens per validation.

RQ1 is answered by the conditional predictive values; RQ2 by paired utility comparisons and the near-corner policies; RQ3 by the validator-cost sensitivity. These answers apply to the simulated error mechanisms and do not establish a general rule against collaboration.

6.1. Recommendations

- Do not assume that structural change signals are sufficient to route collaboration; test the predictive value of the signal against the agent actually used.
- When a validator is available and cheap, prefer failure-triggered collaboration; when it is not, decide between always-single and always-team from the audit's marginal failure removal and the token price.
- Future work should repeat the study with real language-model agents, on REALM-Bench scheduling tasks, and with features that reflect the agent's own uncertainty in addition to the structure of the change.

6.2. Limitations

The agents are algorithmic with injected overlooked checks, not language models, so the quantitative results (failure rates, token counts) do not transfer. Only one task domain with 12-task instances was studied. Utility depends on the token price, which is a modelling choice; we therefore report a grid. The exploratory extensions (cascade, load-dependent errors) were introduced after a pilot, the benchmark REALM-Bench was not used, and the CP-SAT reference is a single solver configuration with one worker and incomplete optimality certification. The new critical-path feature ignores resource-constrained slack and is only a structural proxy. Token counts are assumed charges per check rather than measured model tokens, so their utility is a construct rather than a monetary estimate. Check omissions omit semantic and correlated failure modes. No real-model evaluation was obtained; prompt sensitivity and model-specific generalisation remain unmeasured. Price-grid maxima use test outcomes for descriptive selection and are not independent confirmations. All p-values and intervals concern generated base instances rather than real workloads.

Declarations

Ethics Committee Approval: Not applicable: synthetic tasks and simulated data; no human participants or personal data were involved.

Author Contributions: The authors take collective responsibility for this manuscript.

Conflict of Interest: The authors declare no conflict of interest.

Funding: This research received no external funding.

Data and Code Availability: Code, data and generated results: https://github.com/moslem-mohseni/dependency-change-collaboration-trigger. Supplementary material: supplementary_p2.zip; also available from the corresponding author.

References

Banerjee, P., & Chadha, A. (2026). Routed graph handoff: Adaptive format selection for multi-agent LLM delegation. In *Findings of the Association for Computational Linguistics: EMNLP 2026* (pp. 42756–42765). Association for Computational Linguistics. https://aclanthology.org/2026.findings-emnlp.2008/

Blazewicz, J., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1983). Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics*, *5*(1), 11–24. https://doi.org/10.1016/0166-218X(83)90012-4

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? *arXiv*. https://arxiv.org/abs/2503.13657

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1), 1–26. https://doi.org/10.1214/aos/1176344552

Field, C. A., & Welsh, A. H. (2007). Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *69*(3), 369–390. https://doi.org/10.1111/j.1467-9868.2007.00593.x

Geng, L., & Chang, E. Y. (2025). REALM-Bench: A benchmark for evaluating multi-agent systems on real-world, dynamic planning and scheduling tasks. *arXiv*. https://arxiv.org/abs/2502.18836

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70. https://www.jstor.org/stable/4615733

Hong, Y. (2026). Learning how much to collaborate: Difficulty-aware topology selection for multi-agent code generation. *arXiv*. https://arxiv.org/abs/2609.13890v1

Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., & Liu, X. (2026). Towards a science of scaling agent systems. *arXiv*. https://arxiv.org/abs/2512.08296v3

Mansoor, I. K., Phadke, A., & Rana, P. (2026). Verified tool calls improve LLM agent reliability under non-atomic failures. *arXiv*. https://arxiv.org/abs/2608.02645

Perron, L., & Furnon, V. (2025). *OR-Tools* (Version 9.15) [Computer software]. Google. https://developers.google.com/optimization

Vieira, G. E., Herrmann, J. W., & Lin, E. (2003). Rescheduling manufacturing systems: A framework of strategies, policies, and methods. *Journal of Scheduling*, *6*(1), 39–62. https://doi.org/10.1023/A:1022235519958